

■ Resumen SQL

Base de Datos · 75.15 / 95.05 · UBA FIUBA

Este resumen cubre todos los temas de SQL que entran en el parcial, organizados por concepto. Para cada uno encontrarás: **qué hace**, **cuándo usarlo** y un **ejemplo práctico**.

1 · Creación y Estructura

CREATE SCHEMA

¿Qué hace?

Crea un nuevo esquema (contenedor lógico) dentro de la base de datos, agrupando tablas, vistas y otros objetos.

¿Cuándo usarlo?

Cuando necesitás separar lógicamente objetos de distintas aplicaciones dentro de un mismo motor de base de datos.

Ejemplo:

```
CREATE SCHEMA aplicacion_futbol AUTHORIZATION 'juanperez';
```

CREATE TABLE

¿Qué hace?

Define una nueva tabla indicando sus columnas, tipos de datos y restricciones (PRIMARY KEY, FOREIGN KEY, NOT NULL, DEFAULT, CHECK, UNIQUE).

¿Cuándo usarlo?

Al iniciar el diseño físico: cada entidad del modelo ER se convierte en una tabla.

Ejemplo:

```
CREATE TABLE jugadores (  
  
  DNI INT,  
  
  nombre VARCHAR(32),  
  
  apellido VARCHAR(32),  
  
  equipo VARCHAR(32),  
  
  PRIMARY KEY (DNI),  
  
  CONSTRAINT equipo_fk  
  
  FOREIGN KEY (equipo) REFERENCES equipos(nombre)  
  
  ON DELETE CASCADE ON UPDATE SET NULL  
  
);
```

CREATE DOMAIN

¿Qué hace?

Crea un tipo de dato personalizado basado en uno existente, opcionalmente con una restricción CHECK.

¿Cuándo usarlo?

Cuando el mismo tipo con las mismas restricciones se usa en muchas tablas: cambiando el dominio, el cambio se propaga a todas.

Ejemplo:

```
CREATE DOMAIN tipo_fundacion AS INT  
  
CHECK (VALUE > 1800 AND VALUE < 2025);
```

ALTER TABLE

¿Qué hace?

Modifica la estructura de una tabla existente: agrega, elimina o cambia columnas.

¿Cuándo usarlo?

Cuando el esquema evoluciona y necesitas agregar/quitar/modificar columnas sin recrear la tabla.

Ejemplo:

```
-- Agregar columna

ALTER TABLE alumnos ADD COLUMN fecha_nacimiento DATE;

-- Quitar columna

ALTER TABLE alumnos DROP COLUMN fecha_nacimiento;

-- Cambiar tipo

ALTER TABLE alumnos ALTER COLUMN nombre VARCHAR;
```

DROP TABLE / DROP SCHEMA

¿Qué hace?

Elimina permanentemente una tabla o un esquema completo. Con CASCADE también borra objetos que dependan de él.

¿Cuándo usarlo?

Con extremo cuidado: para limpiar objetos que ya no se necesitan.

- ¡Irreversible! Hacer backup antes.

Ejemplo:

```
DROP TABLE alumnos; -- falla si hay FK que la referencian

DROP TABLE alumnos CASCADE; -- borra también las referencias

DROP SCHEMA aplicacion_futbol;
```

2 · Consultas Básicas

SELECT ... FROM ... WHERE

¿Qué hace?

Recupera filas de una o más tablas aplicando proyección (SELECT), origen (FROM) y filtro (WHERE). Por defecto devuelve duplicados (ALL).

¿Cuándo usarlo?

Es la consulta más básica y fundamental: cualquier lectura de datos.

Ejemplo:

```
-- Todos los equipos fundados después de 1980

SELECT *

FROM equipos

WHERE fundacion > 1980;

-- Con alias para desambiguar columnas

SELECT j.*, e.nombre AS nombre_equipo, e.division

FROM jugadores AS j, equipos AS e

WHERE j.equipo = e.nombre;
```

SELECT DISTINCT

¿Qué hace?

Elimina filas duplicadas del resultado.

¿Cuándo usarlo?

Cuando el mismo valor puede aparecer varias veces y sólo te interesa saber si existe, no cuántas veces.

Ejemplo:

```
SELECT DISTINCT nombre

FROM jugadores

WHERE equipo = 'San Martín';
```

ORDER BY

¿Qué hace?

Ordena el resultado por uno o más atributos. Por defecto ascendente (ASC); se puede especificar DESC.

¿Cuándo usarlo?

Siempre que necesitás presentar resultados en un orden determinado.

Ejemplo:

```
SELECT nombre, apellido
FROM jugadores
WHERE equipo = 'San Martín'
ORDER BY apellido DESC;
```

LIKE (% y _)

¿Qué hace?

Filtra cadenas de texto por patrón. % reemplaza 0 o más caracteres; _ reemplaza exactamente uno.

¿Cuándo usarlo?

Búsquedas parciales de texto: nombres que empiezan con algo, terminan con algo, o tienen cierta longitud.

Ejemplo:

```
-- Equipos que contengan 'Martín' en el nombre
SELECT * FROM jugadores WHERE equipo LIKE '%Martín%';

-- Apellidos de 8 caracteres terminados en 'ez'
SELECT * FROM jugadores WHERE apellido LIKE '____ez';
```

UNION / UNION ALL

¿Qué hace?

UNION: combina resultados de dos SELECT eliminando duplicados. **UNION ALL**: los mantiene.

¿Cuándo usarlo?

Para unir conjuntos de filas de distintas consultas que tienen la misma estructura de columnas.

Ejemplo:

```
SELECT nombre, apellido FROM jugadores WHERE equipo = 'San Martín'

UNION

SELECT nombre, apellido FROM jugadores WHERE equipo = 'Belgrano';
```

EXCEPT / INTERSECT

¿Qué hace?

EXCEPT: devuelve las filas del primer SELECT que NO están en el segundo. **INTERSECT:** devuelve sólo las filas comunes.

¿Cuándo usarlo?

Operaciones de conjuntos: diferencia e intersección entre resultados.

Ejemplo:

```
-- Jugadores de San Martín que NO están en Belgrano

SELECT nombre, apellido FROM jugadores WHERE equipo = 'San Martín'

EXCEPT

SELECT nombre, apellido FROM jugadores WHERE equipo = 'Belgrano';

-- Jugadores que juegan en AMBOS (históricamente)

SELECT nombre, apellido FROM jugadores WHERE equipo = 'San Martín'

INTERSECT

SELECT nombre, apellido FROM jugadores WHERE equipo = 'Belgrano';
```

3 · Consultas Anidadas

Subconsulta en WHERE (IN / NOT IN)

¿Qué hace?

Permite usar el resultado de una consulta interna como conjunto de valores para comparar en la consulta externa con IN o NOT IN.

¿Cuándo usarlo?

Cuando necesitas filtrar filas basándote en si un valor pertenece o no a un conjunto calculado dinámicamente.

Ejemplo:

```
-- Alumnos que también son profesores

SELECT *

FROM alumnos

WHERE dni IN (SELECT dni FROM profesores);
```

ANY / ALL (con subconsulta)

¿Qué hace?

ANY / SOME: verdadero si la condición se cumple para AL MENOS UN elemento del subconjunto. **ALL:** verdadero sólo si se cumple para TODOS.

¿Cuándo usarlo?

Comparar un valor contra un conjunto de valores retornado por una subconsulta.

Ejemplo:

```
-- El/los profesores con mayor sueldo (igual al máximo)

SELECT *

FROM profesores

WHERE sueldo >= ALL (SELECT sueldo FROM profesores);

-- Profesores con sueldo mayor que algún ayudante

SELECT *

FROM profesores

WHERE sueldo > ANY (SELECT sueldo FROM ayudantes);
```

Subconsultas correlacionadas

¿Qué hace?

La subconsulta hace referencia a un atributo de la consulta externa. Se re-evalúa para cada fila de la consulta externa.

¿Cuándo usarlo?

Cuando la condición de la subconsulta depende de la fila actual de la consulta externa. Más expresivas que las no correlacionadas, pero potencialmente más lentas.

Ejemplo:

```
-- Alumnos aprobados en Base de Datos (nota >= 7)

SELECT a.nombre, a.apellido

FROM alumnos AS a

WHERE a.padron IN (

SELECT na.padron

FROM notas_alumnos AS na

WHERE a.padron = na.padron -- correlación

AND na.materia = 'Base de Datos'

AND na.nota_final >= 7

);
```


EXISTS / NOT EXISTS

¿Qué hace?

Devuelve TRUE si la subconsulta retorna al menos una fila (EXISTS) o ninguna (NOT EXISTS). No importa qué columnas devuelva la subconsulta.

¿Cuándo usarlo?

Verificar existencia: más eficiente que IN en muchos casos porque el motor puede parar en la primera fila encontrada.

Ejemplo:

```
-- Alumnos que aprobaron Base de Datos

SELECT a.nombre, a.apellido

FROM alumnos AS a

WHERE EXISTS (

  SELECT *

  FROM notas_alumnos AS na

  WHERE a.padron = na.padron

  AND na.materia = 'Base de Datos'

  AND na.nota_final >= 7

);
```

UNIQUE (subconsulta)

¿Qué hace?

Devuelve TRUE si la subconsulta NO tiene filas duplicadas.

¿Cuándo usarlo?

Cuando querés verificar que una subconsulta devuelve valores únicos. No está implementado en todos los SGBD (Postgres no lo soporta nativamente).

- Puede no estar disponible en PostgreSQL.

Ejemplo:

```
-- Alumnos que aprobaron exactamente 1 materia este cuatrimestre

SELECT a.nombre, a.apellido

FROM alumnos AS a

WHERE UNIQUE (

  SELECT na.padron

  FROM notas_alumnos AS na

  WHERE a.padron = na.padron

  AND na.nota_final >= 7

  AND na.anio = 2024

  AND na.cuatrimestre = 2

);
```

4 · JOINS (Juntas)

INNER JOIN (JOIN por defecto)

¿Qué hace?

Devuelve sólo las filas que tienen coincidencia en AMBAS tablas según la condición ON. Las filas sin match son descartadas.

¿Cuándo usarlo?

Cuando sólo te interesan los registros que existen en ambas tablas (la intersección con condición).

Ejemplo:

```
SELECT d.nombre, d.apellido, d.legajo, m.codigo  
  
FROM docentes AS d  
  
JOIN materias AS m  
  
ON d.legajo = m.docente  
  
WHERE m.inicio > 2002;
```

NATURAL JOIN

¿Qué hace?

Une dos tablas por todas las columnas que comparten el mismo nombre, sin necesidad de especificar la condición ON.

¿Cuándo usarlo?

Atajos cuando los atributos de unión tienen el mismo nombre en ambas tablas. Usarlo con cuidado: si se agregan columnas con el mismo nombre por coincidencia, el resultado cambia.

Ejemplo:

```
SELECT d.nombre, d.apellido, d.legajo, m.codigo  
  
FROM docentes AS d  
  
NATURAL JOIN materias m  
  
WHERE m.inicio > 2002;
```

LEFT OUTER JOIN

¿Qué hace?

Devuelve TODAS las filas de la tabla izquierda, más las coincidencias de la tabla derecha. Donde no hay match, las columnas de la derecha quedan NULL.

¿Cuándo usarlo?

Cuando querés todos los registros de la tabla principal aunque no tengan registros relacionados en la otra tabla. Ej: todos los docentes, tengan o no materias asignadas.

Ejemplo:

```
SELECT d.nombre, d.apellido, d.legajo, m.codigo  
  
FROM docentes AS d  
  
LEFT OUTER JOIN materias AS m  
  
ON d.legajo = m.docente  
  
WHERE m.inicio > 2002;
```

RIGHT OUTER JOIN

¿Qué hace?

Devuelve TODAS las filas de la tabla derecha, más las coincidencias de la tabla izquierda. Donde no hay match, las columnas de la izquierda quedan NULL.

¿Cuándo usarlo?

Cuando querés todos los registros de la segunda tabla aunque no tengan registros relacionados en la primera. Ej: todas las materias, tengan o no docente asignado.

Ejemplo:

```
SELECT d.nombre, d.apellido, d.legajo, m.codigo  
  
FROM docentes AS d  
  
RIGHT OUTER JOIN materias AS m  
  
ON d.legajo = m.docente  
  
WHERE m.inicio > 2002;
```

FULL OUTER JOIN

¿Qué hace?

Combina LEFT y RIGHT: devuelve TODAS las filas de ambas tablas. Donde no hay coincidencia en alguno de los lados, pone NULL.

¿Cuándo usarlo?

Cuando necesitas ver todos los registros de ambas tablas, independientemente de si tienen o no match entre sí.

Ejemplo:

```
SELECT d.nombre, d.apellido, d.legajo, m.codigo  
  
FROM docentes AS d  
  
FULL OUTER JOIN materias AS m  
  
ON d.legajo = m.docente  
  
WHERE m.inicio > 2002;
```

5 · Funciones de Agregación

COUNT

¿Qué hace?

COUNT(*): cuenta todas las filas (incluyendo NULLs). **COUNT(columna)**: cuenta sólo las filas donde esa columna NO es NULL.

¿Cuándo usarlo?

Para saber cuántos registros hay, o cuántos tienen un campo completado.

Ejemplo:

```
SELECT COUNT(*) AS total_filas FROM alumnos;

SELECT COUNT(padron) AS con_padron FROM alumnos;
```

SUM / AVG / MAX / MIN

¿Qué hace?

SUM: suma los valores. **AVG**: promedio. **MAX/MIN**: máximo/mínimo. Todas ignoran NULLs.

¿Cuándo usarlo?

Cálculos estadísticos sobre un conjunto de filas.

Ejemplo:

```
SELECT

SUM(sueldo) AS total_sueldos,

AVG(sueldo) AS promedio,

MAX(sueldo) AS mayor_sueldo,

MIN(sueldo) AS menor_sueldo

FROM docentes;
```

GROUP BY

¿Qué hace?

Agrupar filas que tienen el mismo valor en las columnas indicadas, permitiendo aplicar funciones de agregación a cada grupo por separado. Los atributos agrupados deben aparecer en el SELECT.

¿Cuándo usarlo?

Cuando querés calcular una función de agregación para distintas categorías o grupos (ej: promedio de sueldo por cargo).

Ejemplo:

```
-- Contar docentes por cargo

SELECT cargo, COUNT(*) AS cantidad

FROM docentes

GROUP BY cargo;

-- Promedio de sueldo por cargo

SELECT cargo, AVG(sueldo) AS promedio_sueldo

FROM docentes

GROUP BY cargo;
```

HAVING

¿Qué hace?

Filtra los GRUPOS generados por GROUP BY según una condición. Es el equivalente del WHERE pero aplicado después de la agregación.

¿Cuándo usarlo?

Cuando querés filtrar por el resultado de una función de agregación (lo que no se puede hacer con WHERE, que filtra filas individuales antes de agrupar).

Ejemplo:

```
-- Solo cargos con más de 5 docentes

SELECT cargo, COUNT(*) AS cantidad

FROM docentes

GROUP BY cargo

HAVING COUNT(*) > 5;


-- Excluir ayudantes del conteo por cargo

SELECT cargo, COUNT(*) AS cantidad

FROM docentes

GROUP BY cargo

HAVING cargo <> 'Ayudante 1' AND cargo <> 'Ayudante 2';
```


6 · WITH y WITH RECURSIVE

WITH (CTE — Common Table Expression)

¿Qué hace?

Crea una tabla temporal con nombre que puede usarse en la consulta principal. Mejora la legibilidad y evita subconsultas repetidas.

¿Cuándo usarlo?

Cuando una subconsulta se usa más de una vez, o cuando querés dividir una consulta compleja en pasos más legibles.

Ejemplo:

```
WITH docentes_no_ayudantes(cargo) AS (  
  
  SELECT cargo  
  
  FROM docentes  
  
  WHERE cargo <> 'Ayudante 1'  
  
  AND cargo <> 'Ayudante 2'  
  
  )  
  
  SELECT cargo, COUNT(*)  
  
  FROM docentes_no_ayudantes  
  
  GROUP BY cargo;
```

WITH RECURSIVE

¿Qué hace?

Permite definir una CTE que se referencia a sí misma, útil para recorrer estructuras jerárquicas (árboles, grafos) almacenadas en tablas.

¿Cuándo usarlo?

Jerarquías: organigramas, categorías anidadas, relaciones jefe-empleado, cadenas feudales, rutas de grafos.

Ejemplo:

```
-- Obtener todos los señores feudales de Tristan hacia arriba

WITH RECURSIVE feudo AS (

  -- Caso base: el señor directo de Tristan

  SELECT vasallo_de AS v

  FROM vasallos

  WHERE nombre = 'Tristan'

  UNION

  -- Paso recursivo: subir un nivel más

  SELECT vasallos.vasallo_de

  FROM feudo

  JOIN vasallos ON feudo.v = vasallos.nombre

)

SELECT * FROM feudo;
```

CASE WHEN ... THEN ... END

¿Qué hace?

Expresión condicional que devuelve distintos valores según qué condición se cumpla. Funciona como un if-else dentro de SQL. Se puede usar en SELECT, UPDATE, INSERT.

¿Cuándo usarlo?

Mostrar etiquetas distintas según un código, calcular valores condicionales, clasificar registros.

Ejemplo:

```
SELECT nombre, apellido,  
  
CASE  
  
  WHEN cargo = 'Profesor' THEN 'Profesor Adjunto'  
  
  WHEN cargo = 'JTP' THEN 'Jefe de Trabajos Practicos'  
  
  WHEN cargo = 'Ayudante 1' THEN 'Ayudante de Primera'  
  
  WHEN cargo = 'Ayudante 2' THEN 'Ayudante de Segunda'  
  
  ELSE 'Cargo desconocido'  
  
END AS cargo_completo  
  
FROM docentes;
```

8 · Assertions y Triggers

ASSERTION

¿Qué hace?

Restricción global que debe cumplirse en CUALQUIER estado de la base de datos, sin importar qué tabla cambie. Se define con CREATE ASSERTION ... CHECK (condición).

¿Cuándo usarlo?

Cuando una restricción involucra más de una tabla o no puede expresarse como CHECK de columna/fila. Usar sólo si no alcanza con CHECK en tabla; son costosas para el SGBD.

- Preferir CHECK en tabla/dominio si es posible; son más eficientes.

Ejemplo:

```
CREATE ASSERTION sueldo_minimo_docente

CHECK (

NOT EXISTS (

SELECT *

FROM docentes d

WHERE d.sueldo < 60000

)

);
```

TRIGGER

¿Qué hace?

Procedimiento que se ejecuta automáticamente cuando ocurre un evento (INSERT, UPDATE, DELETE) sobre una tabla, opcionalmente bajo una condición. Tiene 3 partes: Evento · Condición · Acción.

¿Cuándo usarlo?

Auditoría, validaciones complejas, propagación automática de cambios (ej: registrar en un log cada vez que se inserta un sueldo muy alto).

Ejemplo:

```
CREATE TRIGGER aviso_salario_alto

BEFORE INSERT OR UPDATE OF sueldo ON docentes

FOR EACH ROW

WHEN (NEW.sueldo > 1000000)

INSERT INTO salario_alto

VALUES (NEW.nombre, NEW.apellido, NEW.legajo, NEW.sueldo);
```

9 · Vistas (VIEW)

CREATE VIEW

¿Qué hace?

Crea una tabla virtual derivada de una consulta. No almacena datos (es virtual por defecto): cada vez que se consulta, re-ejecuta la consulta subyacente. Se puede crear una vista sobre otra vista.

¿Cuándo usarlo?

Simplificar consultas complejas, controlar permisos (mostrar sólo ciertas columnas a ciertos usuarios), encapsular lógica de negocio.

Ejemplo:

```
CREATE VIEW alumnos_y_docentes(nombre, apellido, id) AS

SELECT a.nombre, a.apellido, a.padron FROM alumnos a

UNION

SELECT d.nombre, d.apellido, d.legajo FROM docentes d;

-- Consultar la vista como si fuera una tabla

SELECT * FROM alumnos_y_docentes WHERE nombre = 'Juan';

-- Eliminar la vista

DROP VIEW alumnos_y_docentes;
```

10 · DML: INSERT · UPDATE · DELETE

INSERT INTO

¿Qué hace?

Agrega nuevas filas a una tabla. Puede insertar valores literales o el resultado de un SELECT.

¿Cuándo usarlo?

Alta de registros nuevos.

Ejemplo:

```
-- Valores literales

INSERT INTO jugadores (nombre, apellido, equipo)

VALUES ('Juan', 'Gomez', 'Estudiantes de Catamarca');

-- Desde una consulta

INSERT INTO jugadores_de_primera (nombre, apellido, equipo)

SELECT j.nombre, j.apellido, j.equipo

FROM jugadores j

JOIN equipos e ON j.equipo = e.nombre

WHERE e.division = 'primera';
```

UPDATE

¿Qué hace?

Modifica valores de filas existentes en una tabla, filtrando con WHERE.

¿Cuándo usarlo?

Cambios en datos existentes. Sin WHERE afecta TODAS las filas.

- Sin cláusula WHERE se actualizan TODAS las filas.

Ejemplo:

```
UPDATE jugadores

SET equipo = 'San Martin de Santiago del Estero'

WHERE equipo = 'San Martin';
```

DELETE

¿Qué hace?

Elimina filas de una tabla según una condición WHERE.

¿Cuándo usarlo?

Baja de registros. Sin WHERE elimina todos los registros de la tabla.

- Sin cláusula WHERE se borran TODAS las filas.

Ejemplo:

```
DELETE FROM jugadores  
  
WHERE equipo = 'San Martin';
```

11 · Funciones de SQL

SUBSTRING / UPPER / char_length

¿Qué hace?

SUBSTRING(str, inicio, largo): recorta una cadena. **UPPER(str)**: convierte a mayúsculas. **char_length(str)**: devuelve la cantidad de caracteres.

¿Cuándo usarlo?

Manipulación de cadenas de texto en el resultado de una consulta.

Ejemplo:

```
SELECT SUBSTRING(apellido, 1, 8) AS abbrev FROM alumnos;  
  
SELECT UPPER(nombre) AS mayusc FROM alumnos;  
  
SELECT char_length(nombre), nombre FROM alumnos;
```

EXTRACT / COALESCE / CAST

¿Qué hace?

EXTRACT(parte FROM fecha): extrae año, mes, día, hora, etc. **COALESCE**(v1, v2, ...): devuelve el primer valor no-NULL. **CAST**(valor AS tipo): convierte entre tipos de datos.

¿Cuándo usarlo?

Transformaciones de fechas, manejo de NULLs con valor por defecto, conversión de tipos.

Ejemplo:

```
-- Extraer año de nacimiento

SELECT EXTRACT(YEAR FROM nacimiento) FROM alumnos;

-- Si nacimiento es NULL, mostrar fecha por defecto

SELECT COALESCE(nacimiento, '1900-01-01') FROM alumnos;

-- Convertir texto a timestamp

SELECT CAST('2024-08-25' AS TIMESTAMP);
```

OFFSET / FETCH (Paginación)

¿Qué hace?

Permite saltarse N filas (OFFSET) y traer sólo las próximas M (FETCH FIRST M ROWS ONLY). Útil para implementar paginación en aplicaciones.

¿Cuándo usarlo?

APIs y aplicaciones web que muestran resultados por páginas.

Ejemplo:

```
-- Página 6 (filas 51 a 60)

SELECT *

FROM alumnos

OFFSET 50 ROWS

FETCH FIRST 10 ROWS ONLY;
```


IS NULL / IS NOT NULL (Lógica trivalente)

¿Qué hace?

SQL usa lógica de tres valores: TRUE, FALSE, UNKNOWN. Los NULLs no se comparan con = sino con IS NULL o IS NOT NULL. Cualquier comparación con NULL devuelve UNKNOWN.

¿Cuándo usarlo?

Verificar si un campo tiene o no valor asignado. Recordar que NULL = NULL devuelve UNKNOWN (no TRUE).

Ejemplo:

```
-- Profesores sin legajo asignado

SELECT * FROM profesores WHERE legajo IS NULL;

-- Alumnos con email cargado

SELECT * FROM alumnos WHERE email IS NOT NULL;
```